

★ 21.2-7

Suppose that the edge weights in a graph are uniformly distributed over the half-open interval $[0, 1)$. Which algorithm, Kruskal's or Prim's, can you make run faster?

★ 21.2-8

Suppose that a graph G has a minimum spanning tree already computed. How quickly can you update the minimum spanning tree upon adding a new vertex and incident edges to G ?

Problems
21-1 Second-best minimum spanning tree

Let $G = (V, E)$ be an undirected, connected graph whose weight function is $w : E \rightarrow \mathbb{R}$, and suppose that $|E| \geq |V|$ and all edge weights are distinct.

We define a second-best minimum spanning tree as follows. Let $\mathcal{T}$ be the set of all spanning trees of G , and let T be a minimum spanning tree of G . Then a *second-best minimum spanning tree* is a spanning tree T' such that $w(T') = \min \{w(T'') : T'' \in \mathcal{T} - \{T\}\}$.

- Show that the minimum spanning tree is unique, but that the second-best minimum spanning tree need not be unique.
- Let T be the minimum spanning tree of G . Prove that G contains some edge $(u, v) \in T$ and some edge $(x, y) \notin T$ such that $(T - \{(u, v)\}) \cup \{(x, y)\}$ is a second-best minimum spanning tree of G .
- Now let T be any spanning tree of G and, for any two vertices $u, v \in V$, let $\max[u, v]$ denote an edge of maximum weight on the unique simple path between u and v in T . Describe an $O(V^2)$ -time algorithm that, given T , computes $\max[u, v]$ for all $u, v \in V$.
- Give an efficient algorithm to compute the second-best minimum spanning tree of G .

21-2 Minimum spanning tree in sparse graphs

For a very sparse connected graph $G = (V, E)$, it is possible to further improve upon the $O(E + V \lg V)$ running time of Prim's algorithm with a Fibonacci heap by preprocessing G to decrease the number of vertices before running Prim's algorithm. In particular, for each vertex u , choose the minimum-weight edge (u, v)

incident on u , and put (u, v) into the minimum spanning tree under construction. Then, contract all chosen edges (see Section B.4). Rather than contracting these edges one at a time, first identify sets of vertices that are united into the same new vertex. Then create the graph that would have resulted from contracting these edges one at a time, but do so by “renaming” edges according to the sets into which their endpoints were placed. Several edges from the original graph might be renamed the same as each other. In such a case, only one edge results, and its weight is the minimum of the weights of the corresponding original edges.

Initially, set the minimum spanning tree T being constructed to be empty, and for each edge $(u, v) \in E$, initialize the two attributes $(u, v).orig = (u, v)$ and $(u, v).c = w(u, v)$. Use the *orig* attribute to reference the edge from the initial graph that is associated with an edge in the contracted graph. The *c* attribute holds the weight of an edge, and as edges are contracted, it is updated according to the above scheme for choosing edge weights. The procedure MST-REDUCE on the facing page takes inputs G and T , and it returns a contracted graph G' with updated attributes *orig'* and *c'*. The procedure also accumulates edges of G into the minimum spanning tree T .

- a.* Let T be the set of edges returned by MST-REDUCE, and let A be the minimum spanning tree of the graph G' formed by the call MST-PRIM(G', c', r), where c' is the weight attribute on the edges of $G'.E$ and r is any vertex in $G'.V$. Prove that $T \cup \{(x, y).orig' : (x, y) \in A\}$ is a minimum spanning tree of G .
- b.* Argue that $|G'.V| \leq |V|/2$.
- c.* Show how to implement MST-REDUCE so that it runs in $O(E)$ time. (*Hint:* Use simple data structures.)
- d.* Suppose that you run k phases of MST-REDUCE, using the output G' produced by one phase as the input G to the next phase and accumulating edges in T . Argue that the overall running time of the k phases is $O(kE)$.
- e.* Suppose that after running k phases of MST-REDUCE, as in part (d), you run Prim's algorithm by calling MST-PRIM(G', c', r), where G' , with weight attribute c' , is returned by the last phase and r is any vertex in $G'.V$. Show how to pick k so that the overall running time is $O(E \lg \lg V)$. Argue that your choice of k minimizes the overall asymptotic running time.
- f.* For what values of $|E|$ (in terms of $|V|$) does Prim's algorithm with preprocessing asymptotically beat Prim's algorithm without preprocessing?

```

MST-REDUCE( $G, T$ )
1  for each vertex  $v \in G.V$ 
2       $v.mark = \text{FALSE}$ 
3      MAKE-SET( $v$ )
4  for each vertex  $u \in G.V$ 
5      if  $u.mark == \text{FALSE}$ 
6          choose  $v \in G.Adj[u]$  such that  $(u, v).c$  is minimized
7          UNION( $u, v$ )
8           $T = T \cup \{(u, v).orig\}$ 
9           $u.mark = \text{TRUE}$ 
10          $v.mark = \text{TRUE}$ 
11   $G'.V = \{\text{FIND-SET}(v) : v \in G.V\}$ 
12   $G'.E = \emptyset$ 
13  for each edge  $(x, y) \in G.E$ 
14       $u = \text{FIND-SET}(x)$ 
15       $v = \text{FIND-SET}(y)$ 
16      if  $u \neq v$ 
17          if  $(u, v) \notin G'.E$ 
18               $G'.E = G'.E \cup \{(u, v)\}$ 
19               $(u, v).orig' = (x, y).orig$ 
20               $(u, v).c' = (x, y).c$ 
21          elseif  $(x, y).c < (u, v).c'$ 
22               $(u, v).orig' = (x, y).orig$ 
23               $(u, v).c' = (x, y).c$ 
24  construct adjacency lists  $G'.Adj$  for  $G'$ 
25  return  $G'$  and  $T$ 

```

21-3 *Alternative minimum-spanning-tree algorithms*

Consider the three algorithms MAYBE-MST-A, MAYBE-MST-B, and MAYBE-MST-C on the next page. Each one takes a connected graph and a weight function as input and returns a set of edges T . For each algorithm, either prove that T is a minimum spanning tree or prove that T is not necessarily a minimum spanning tree. Also describe the most efficient implementation of each algorithm, regardless of whether it computes a minimum spanning tree.

21-4 *Bottleneck spanning tree*

A *bottleneck spanning tree* T of an undirected graph G is a spanning tree of G whose largest edge weight is minimum over all spanning trees of G . The value of the bottleneck spanning tree is the weight of the maximum-weight edge in T .

MAYBE-MST-A(G, w)

```

1  sort the edges into monotonically decreasing order of edge weights  $w$ 
2   $T = E$ 
3  for each edge  $e$ , taken in monotonically decreasing order by weight
4      if  $T - \{e\}$  is a connected graph
5           $T = T - \{e\}$ 
6  return  $T$ 

```

MAYBE-MST-B(G, w)

```

1   $T = \emptyset$ 
2  for each edge  $e$ , taken in arbitrary order
3      if  $T \cup \{e\}$  has no cycles
4           $T = T \cup \{e\}$ 
5  return  $T$ 

```

MAYBE-MST-C(G, w)

```

1   $T = \emptyset$ 
2  for each edge  $e$ , taken in arbitrary order
3       $T = T \cup \{e\}$ 
4      if  $T$  has a cycle  $c$ 
5          let  $e'$  be a maximum-weight edge on  $c$ 
6           $T = T - \{e'\}$ 
7  return  $T$ 

```

a. Argue that a minimum spanning tree is a bottleneck spanning tree.

Part (a) shows that finding a bottleneck spanning tree is no harder than finding a minimum spanning tree. In the remaining parts, you will show how to find a bottleneck spanning tree in linear time.

- b.** Give a linear-time algorithm that, given a graph G and an integer b , determines whether the value of the bottleneck spanning tree is at most b .
- c.** Use your algorithm for part (b) as a subroutine in a linear-time algorithm for the bottleneck-spanning-tree problem. (*Hint:* You might want to use a subroutine that contracts sets of edges, as in the MST-REDUCE procedure described in Problem 21-2.)

Chapter notes

Tarjan [429] surveys the minimum-spanning-tree problem and provides excellent advanced material. Graham and Hell [198] compiled a history of the minimum-spanning-tree problem.

Tarjan attributes the first minimum-spanning-tree algorithm to a 1926 paper by O. Borůvka. Borůvka's algorithm consists of running $O(\lg V)$ iterations of the procedure MST-REDUCE described in Problem 21-2. Kruskal's algorithm was reported by Kruskal [272] in 1956. The algorithm commonly known as Prim's algorithm was indeed invented by Prim [367], but it was also invented earlier by V. Jarník in 1930.

When $|E| = \Omega(V \lg V)$, Prim's algorithm, implemented with a Fibonacci heap, runs in $O(E)$ time. For sparser graphs, using a combination of the ideas from Prim's algorithm, Kruskal's algorithm, and Borůvka's algorithm, together with advanced data structures, Fredman and Tarjan [156] give an algorithm that runs in $O(E \lg^* V)$ time. Gabow, Galil, Spencer, and Tarjan [165] improved this algorithm to run in $O(E \lg \lg^* V)$ time. Chazelle [83] gives an algorithm that runs in $O(E \hat{\alpha}(E, V))$ time, where $\hat{\alpha}(E, V)$ is the functional inverse of Ackermann's function. (See the chapter notes for Chapter 19 for a brief discussion of Ackermann's function and its inverse.) Unlike previous minimum-spanning-tree algorithms, Chazelle's algorithm does not follow the greedy method. Pettie and Ramachandran [356] give an algorithm based on precomputed "MST decision trees" that also runs in $O(E \hat{\alpha}(E, V))$ time.

A related problem is *spanning-tree verification*: given a graph $G = (V, E)$ and a tree $T \subseteq E$, determine whether T is a minimum spanning tree of G . King [254] gives a linear-time algorithm to verify a spanning tree, building on earlier work of Komlós [269] and Dixon, Rauch, and Tarjan [120].

The above algorithms are all deterministic and fall into the comparison-based model described in Chapter 8. Karger, Klein, and Tarjan [243] give a randomized minimum-spanning-tree algorithm that runs in $O(V + E)$ expected time. This algorithm uses recursion in a manner similar to the linear-time selection algorithm in Section 9.3: a recursive call on an auxiliary problem identifies a subset of the edges E' that cannot be in any minimum spanning tree. Another recursive call on $E - E'$ then finds the minimum spanning tree. The algorithm also uses ideas from Borůvka's algorithm and King's algorithm for spanning-tree verification.

Fredman and Willard [158] showed how to find a minimum spanning tree in $O(V + E)$ time using a deterministic algorithm that is not comparison based. Their algorithm assumes that the data are b -bit integers and that the computer memory consists of addressable b -bit words.

Suppose that you need to drive from Oceanside, New York, to Oceanside, California, by the shortest possible route. Your GPS contains information about the entire road network of the United States, including the road distance between each pair of adjacent intersections. How can your GPS determine this shortest route?

One possible way is to enumerate all the routes from Oceanside, New York, to Oceanside, California, add up the distances on each route, and select the shortest. But even disallowing routes that contain cycles, your GPS would need to examine an enormous number of possibilities, most of which are simply not worth considering. For example, a route that passes through Miami, Florida, is a poor choice, because Miami is several hundred miles out of the way.

This chapter and Chapter 23 show how to solve such problems efficiently. The input to a *shortest-paths problem* is a weighted, directed graph $G = (V, E)$, with a weight function $w : E \rightarrow \mathbb{R}$ mapping edges to real-valued weights. The *weight* $w(p)$ of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its constituent edges:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i) .$$

We define the *shortest-path weight* $\delta(u, v)$ from u to v by

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \xrightarrow{p} v\} & \text{if there is a path from } u \text{ to } v , \\ \infty & \text{otherwise .} \end{cases}$$

A *shortest path* from vertex u to vertex v is then defined as any path p with weight $w(p) = \delta(u, v)$.

In the example of going from Oceanside, New York, to Oceanside, California, your GPS models the road network as a graph: vertices represent intersections, edges represent road segments between intersections, and edge weights represent road distances. The goal is to find a shortest path from a given intersection in

Oceanside, New York (say, Brower Avenue and Skillman Avenue) to a given intersection in Oceanside, California (say, Topeka Street and South Horne Street).

Edge weights can represent metrics other than distances, such as time, cost, penalties, loss, or any other quantity that accumulates linearly along a path and that you want to minimize.

The breadth-first-search algorithm from Section 20.2 is a shortest-paths algorithm that works on unweighted graphs, that is, graphs in which each edge has unit weight. Because many of the concepts from breadth-first search arise in the study of shortest paths in weighted graphs, you might want to review Section 20.2 before proceeding.

Variants

This chapter focuses on the *single-source shortest-paths problem*: given a graph $G = (V, E)$, find a shortest path from a given *source vertex* $s \in V$ to every vertex $v \in V$. The algorithm for the single-source problem can solve many other problems, including the following variants.

Single-destination shortest-paths problem: Find a shortest path to a given *destination vertex* t from each vertex v . By reversing the direction of each edge in the graph, you can reduce this problem to a single-source problem.

Single-pair shortest-path problem: Find a shortest path from u to v for given vertices u and v . If you solve the single-source problem with source vertex u , you solve this problem also. Moreover, all known algorithms for this problem have the same worst-case asymptotic running time as the best single-source algorithms.

All-pairs shortest-paths problem: Find a shortest path from u to v for every pair of vertices u and v . Although you can solve this problem by running a single-source algorithm once from each vertex, you often can solve it faster. Additionally, its structure is interesting in its own right. Chapter 23 addresses the all-pairs problem in detail.

Optimal substructure of a shortest path

Shortest-paths algorithms typically rely on the property that a shortest path between two vertices contains other shortest paths within it. (The Edmonds-Karp maximum-flow algorithm in Chapter 24 also relies on this property.) Recall that optimal substructure is one of the key indicators that dynamic programming (Chapter 14) and the greedy method (Chapter 15) might apply. Dijkstra's algorithm, which we shall see in Section 22.3, is a greedy algorithm, and the Floyd-Warshall algorithm, which finds a shortest path between every pair of vertices (see Sec-

tion 23.2), is a dynamic-programming algorithm. The following lemma states the optimal-substructure property of shortest paths more precisely.

Lemma 22.1 (Subpaths of shortest paths are shortest paths)

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from vertex v_0 to vertex v_k and, for any i and j such that $0 \leq i \leq j \leq k$, let $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ be the subpath of p from vertex v_i to vertex v_j . Then, p_{ij} is a shortest path from v_i to v_j .

Proof Decompose path p into $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$, so that $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Now, assume that there is a path p'_{ij} from v_i to v_j with weight $w(p'_{ij}) < w(p_{ij})$. Then, $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p'_{ij}} v_j \xrightarrow{p_{jk}} v_k$ is a path from v_0 to v_k whose weight $w(p_{0i}) + w(p'_{ij}) + w(p_{jk})$ is less than $w(p)$, which contradicts the assumption that p is a shortest path from v_0 to v_k . ■

Negative-weight edges

Some instances of the single-source shortest-paths problem may include edges whose weights are negative. If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source s , then for all $v \in V$, the shortest-path weight $\delta(s, v)$ remains well defined, even if it has a negative value. If the graph contains a negative-weight cycle reachable from s , however, shortest-path weights are not well defined. No path from s to a vertex on the cycle can be a shortest path—you can always find a path with lower weight by following the proposed “shortest” path and then traversing the negative-weight cycle. If there is a negative-weight cycle on some path from s to v , we define $\delta(s, v) = -\infty$.

Figure 22.1 illustrates the effect of negative weights and negative-weight cycles on shortest-path weights. Because there is only one path from s to a (the path $\langle s, a \rangle$), we have $\delta(s, a) = w(s, a) = 3$. Similarly, there is only one path from s to b , and so $\delta(s, b) = w(s, a) + w(a, b) = 3 + (-4) = -1$. There are infinitely many paths from s to c : $\langle s, c \rangle$, $\langle s, c, d, c \rangle$, $\langle s, c, d, c, d, c \rangle$, and so on. Because the cycle $\langle c, d, c \rangle$ has weight $6 + (-3) = 3 > 0$, the shortest path from s to c is $\langle s, c \rangle$, with weight $\delta(s, c) = w(s, c) = 5$, and the shortest path from s to d is $\langle s, c, d \rangle$, with weight $\delta(s, d) = w(s, c) + w(c, d) = 11$. Analogously, there are infinitely many paths from s to e : $\langle s, e \rangle$, $\langle s, e, f, e \rangle$, $\langle s, e, f, e, f, e \rangle$, and so on. Because the cycle $\langle e, f, e \rangle$ has weight $3 + (-6) = -3 < 0$, however, there is no shortest path from s to e . By traversing the negative-weight cycle $\langle e, f, e \rangle$ arbitrarily many times, you can find paths from s to e with arbitrarily large negative weights, and so $\delta(s, e) = -\infty$. Similarly, $\delta(s, f) = -\infty$. Because g is reachable from f , you can also find paths with arbitrarily large negative weights from s to g ,

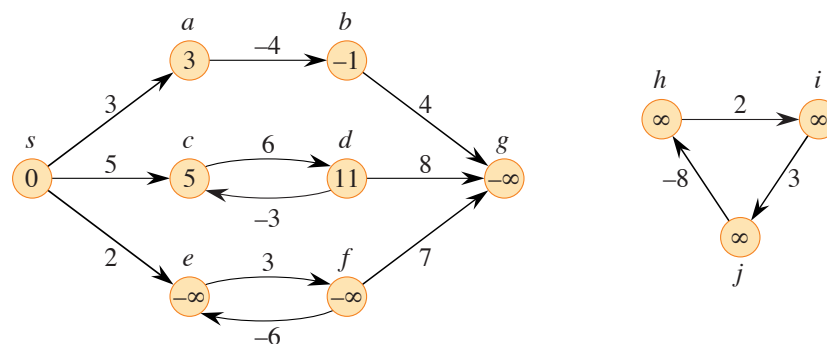


Figure 22.1 Negative edge weights in a directed graph. The shortest-path weight from source s appears within each vertex. Because vertices e and f form a negative-weight cycle reachable from s , they have shortest-path weights of $-\infty$. Because vertex g is reachable from a vertex whose shortest-path weight is $-\infty$, it, too, has a shortest-path weight of $-\infty$. Vertices such as h, i , and j are not reachable from s , and so their shortest-path weights are ∞ , even though they lie on a negative-weight cycle.

and so $\delta(s, g) = -\infty$. Vertices h, i , and j also form a negative-weight cycle. They are not reachable from s , however, and so $\delta(s, h) = \delta(s, i) = \delta(s, j) = \infty$.

Some shortest-paths algorithms, such as Dijkstra's algorithm, assume that all edge weights in the input graph are nonnegative, as in a road network. Others, such as the Bellman-Ford algorithm, allow negative-weight edges in the input graph and produce a correct answer as long as no negative-weight cycles are reachable from the source. Typically, if there is such a negative-weight cycle, the algorithm can detect and report its existence.

Cycles

Can a shortest path contain a cycle? As we have just seen, it cannot contain a negative-weight cycle. Nor can it contain a positive-weight cycle, since removing the cycle from the path produces a path with the same source and destination vertices and a lower path weight. That is, if $p = \langle v_0, v_1, \dots, v_k \rangle$ is a path and $c = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is a positive-weight cycle on this path (so that $v_i = v_j$ and $w(c) > 0$), then the path $p' = \langle v_0, v_1, \dots, v_i, v_{j+1}, v_{j+2}, \dots, v_k \rangle$ has weight $w(p') = w(p) - w(c) < w(p)$, and so p cannot be a shortest path from v_0 to v_k .

That leaves only 0-weight cycles. You can remove a 0-weight cycle from any path to produce another path whose weight is the same. Thus, if there is a shortest path from a source vertex s to a destination vertex v that contains a 0-weight cycle, then there is another shortest path from s to v without this cycle. As long as a shortest path has 0-weight cycles, you can repeatedly remove these cycles from the path until you have a shortest path that is cycle-free. Therefore, without loss of

generality, assume that shortest paths have no cycles, that is, they are simple paths. Since any acyclic path in a graph $G = (V, E)$ contains at most $|V|$ distinct vertices, it also contains at most $|V| - 1$ edges. Assume, therefore, that any shortest path contains at most $|V| - 1$ edges.

Representing shortest paths

It is usually not enough to compute only shortest-path weights. Most applications of shortest paths need to know the vertices on shortest paths as well. For example, if your GPS told you the distance to your destination but not how to get there, it would not be terribly useful. We represent shortest paths similarly to how we represented breadth-first trees in Section 20.2. Given a graph $G = (V, E)$, maintain for each vertex $v \in V$ a **predecessor** $v.\pi$ that is either another vertex or NIL. The shortest-paths algorithms in this chapter set the π attributes so that the chain of predecessors originating at a vertex v runs backward along a shortest path from s to v . Thus, given a vertex v for which $v.\pi \neq \text{NIL}$, the procedure $\text{PRINT-PATH}(G, s, v)$ from Section 20.2 prints a shortest path from s to v .

In the midst of executing a shortest-paths algorithm, however, the π values might not indicate shortest paths. The **predecessor subgraph** $G_\pi = (V_\pi, E_\pi)$ induced by the π values is defined the same for single-source shortest paths as for breadth-first search in equations (20.2) and (20.3) on page 561:

$$\begin{aligned} V_\pi &= \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\} , \\ E_\pi &= \{(v.\pi, v) \in E : v \in V_\pi - \{s\}\} . \end{aligned}$$

We'll prove that the π values produced by the algorithms in this chapter have the property that at termination G_π is a “shortest-paths tree”—informally, a rooted tree containing a shortest path from the source s to every vertex that is reachable from s . A shortest-paths tree is like the breadth-first tree from Section 20.2, but it contains shortest paths from the source defined in terms of edge weights instead of numbers of edges. To be precise, let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$, and assume that G contains no negative-weight cycles reachable from the source vertex $s \in V$, so that shortest paths are well defined. A **shortest-paths tree** rooted at s is a directed subgraph $G' = (V', E')$, where $V' \subseteq V$ and $E' \subseteq E$, such that

1. V' is the set of vertices reachable from s in G ,
2. G' forms a rooted tree with root s , and
3. for all $v \in V'$, the unique simple path from s to v in G' is a shortest path from s to v in G .